In [1]:
```
# 1. Import libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.optimizers import Adam
```

In [2]:
```
# 2. Load and preprocess dataset
path = r"C:\Users\ombah\OmCore\BE Practical\LP-4 (Deep Learning)\LP IV Updated Datasets\Object Detection\Intel Image Classific

datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    validation_split=0.2
)

train_data = datagen.flow_from_directory(
    path,
    target_size=(224, 224),
    batch_size=32,
    class_mode="categorical",
    subset="training"
)

val_data = datagen.flow_from_directory(
    path,
    target_size=(224, 224),
    batch_size=32,
    class_mode="categorical",
    subset="validation"
)
```

```
Found 10109 images belonging to 6 classes.
Found 2523 images belonging to 6 classes.
```

In [3]:
```
# 3. Load pre-trained VGG16 model
base_model = VGG16(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
```

In [4]:
```
    # 4. Freeze base layers
base_model.trainable = False

# 5. Add custom classification layers
x = Flatten()(base_model.output)
x = Dense(256, activation="relu")(x)
x = Dropout(0.5)(x)

output = Dense(
    train_data.num_classes,
    activation="softmax"
)(x)

model = Model(base_model.input, output)
```

In [5]:
```
# 6. Compile the model
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are ins
talled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

In [6]:
```
# 7. Train the model
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=2,
    steps_per_epoch=50,
```

```
    validation_steps=10
)
```

```
Epoch 1/2
50/50 ━━━━━━━━━━━━━━━━━━━━ 170s 3s/step - accuracy: 0.7906 - loss: 6.2172 - val_accuracy: 0.8969 - val_loss: 1.5851
Epoch 2/2
50/50 ━━━━━━━━━━━━━━━━━━━━ 176s 4s/step - accuracy: 0.8425 - loss: 2.4243 - val_accuracy: 0.8906 - val_loss: 0.9008
```

In [7]:
```
# 8. Evaluate the model
loss, accuracy = model.evaluate(val_data)

print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)
```

```
79/79 ━━━━━━━━━━━━━━━━━━━━ 250s 3s/step - accuracy: 0.8962 - loss: 0.9360
Validation Loss: 0.9360325336456299
Validation Accuracy: 0.8961553573608398
```

In [8]:
```
# 9. Fine-tune the model
base_model.trainable = True

# Freeze all layers except the last 4
for layer in base_model.layers[:-4]:
    layer.trainable = False

# Compile again with a smaller learning rate
model.compile(
    optimizer=Adam(learning_rate=0.00001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# Train the model again
history_fine = model.fit(
    train_data,
    validation_data=val_data,
    epochs=1,
    steps_per_epoch=50,
    validation_steps=10
)
```

```
50/50 ━━━━━━━━━━━━━━━━━━━━ 312s 6s/step - accuracy: 0.8788 - loss: 0.9223 - val_accuracy: 0.9281 - val_loss: 0.4674
```

In [9]:
```python
# 10. Evaluate after fine-tuning
loss, accuracy = model.evaluate(val_data)

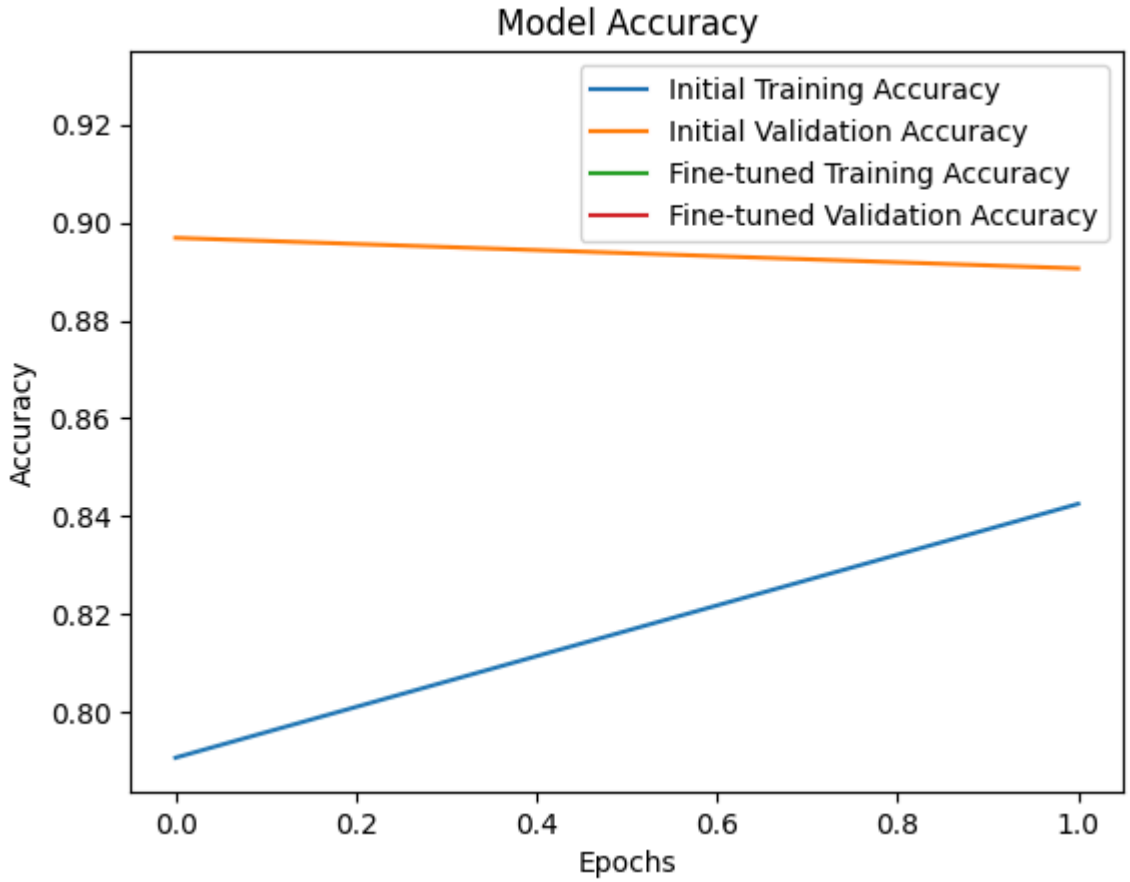
print("Fine-tuned Validation Loss:", loss)
print("Fine-tuned Validation Accuracy:", accuracy)
```

```
79/79 ━━━━━━━━━━━━━━━━━━━━ 323s 4s/step - accuracy: 0.9029 - loss: 0.6495
Fine-tuned Validation Loss: 0.6494627594947815
Fine-tuned Validation Accuracy: 0.9028933644294739
```

In [10]:
```python
# 11. Visualize training accuracy
plt.plot(history.history["accuracy"], label="Initial Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Initial Validation Accuracy")

plt.plot(history_fine.history["accuracy"], label="Fine-tuned Training Accuracy")
plt.plot(history_fine.history["val_accuracy"], label="Fine-tuned Validation Accuracy")

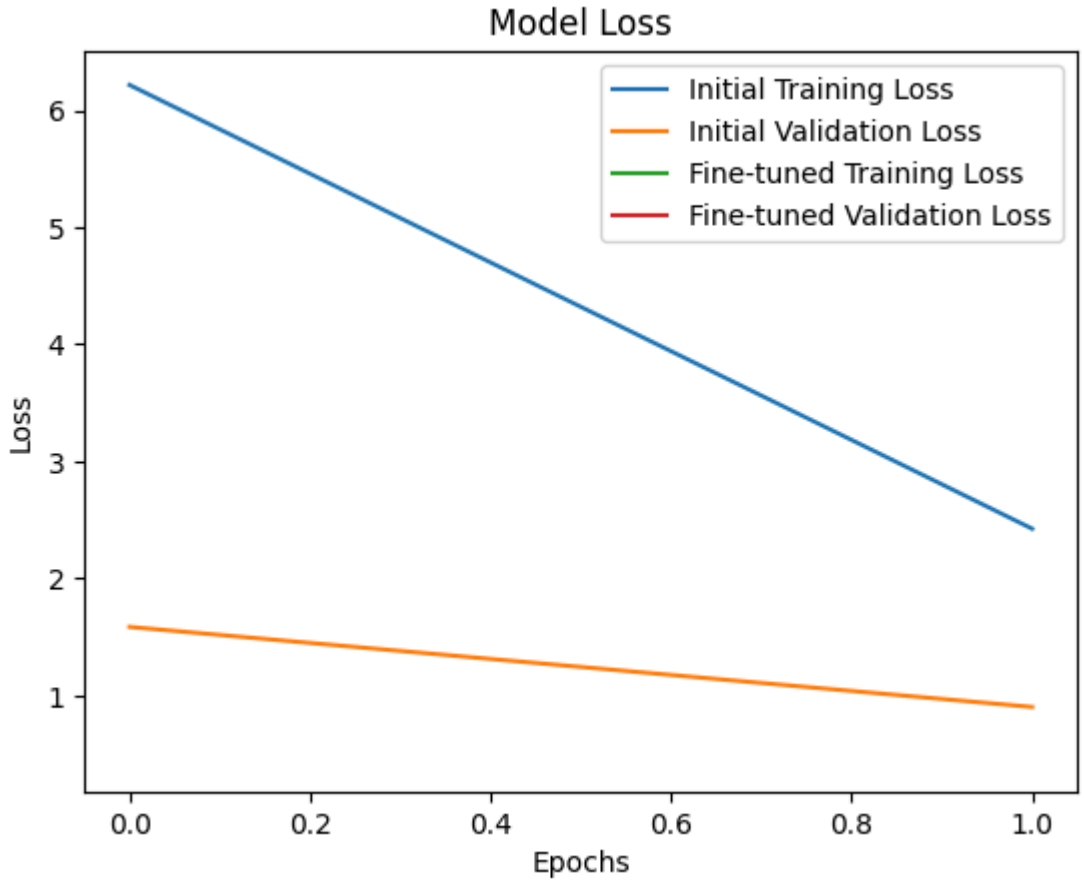
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Model Accuracy")
plt.legend()
plt.show()
```



In [11]:
```
# 12. Visualize training loss
plt.plot(history.history["loss"], label="Initial Training Loss")
plt.plot(history.history["val_loss"], label="Initial Validation Loss")

plt.plot(history_fine.history["loss"], label="Fine-tuned Training Loss")
plt.plot(history_fine.history["val_loss"], label="Fine-tuned Validation Loss")

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Model Loss")
plt.legend()
plt.show()
```



In [16]:
```
# 13. Visualize prediction
img, label = next(val_data)

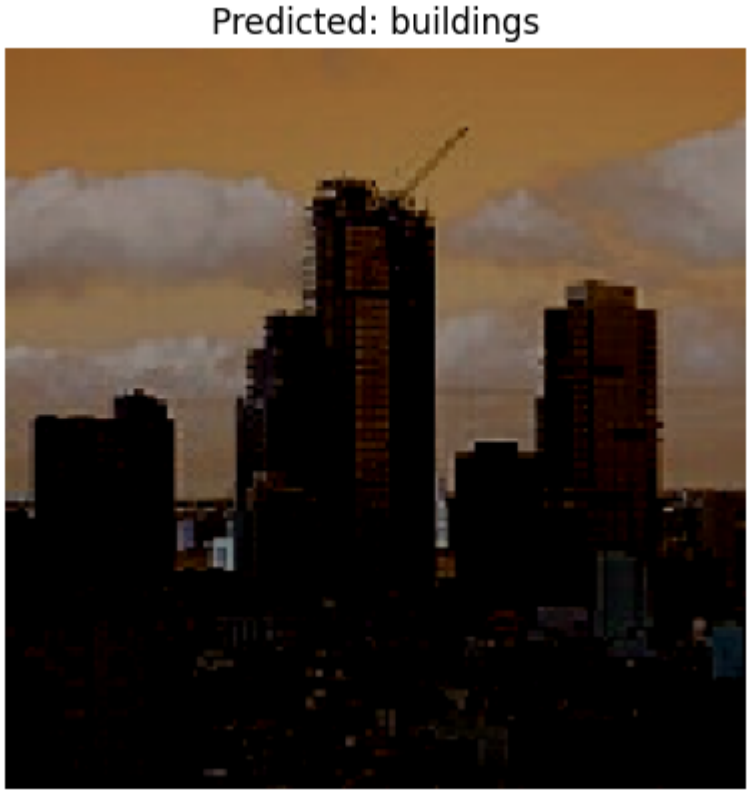
prediction = model.predict(img[:1])
class_index = np.argmax(prediction[0])

class_names = list(train_data.class_indices.keys())

plt.imshow(img[0] / 255.0)
plt.title("Predicted: " + class_names[class_index])
plt.axis("off")
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 160ms/step
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.485
0196..0.5923961].
```



In [ ]: